

TouchDesigner Workshop — Core Curriculum

3 วัน · session-by-session · สำหรับทีม developer + designer · ผู้สอน: Wave Pongruengkiat

นี่คือ แกนกลาง (core) ที่ reuse ได้หลายงาน — รายละเอียดลูกค้า/วัน/เวลา/องค์ประกอบทีม อยู่ในกล่อง

📌 Client Context ด้านล่าง

Learning Outcomes

1. สร้าง interactive system: input (camera/audio) → process (TD) → output ได้เอง
2. เริ่มโปรเจกต์ TD เองได้ + ประเมิน tech ที่เหมาะกับงานของตัวเอง/ทีม
3. ต่อ TD เข้าระบบอื่น (OSC / MQTT / Web / LINE) ได้
4. ทำ mini-project แบบทีม: concept → build → demo
5. รู้วิธีเรียนต่อเอง (docs · community · AI-assisted)

Principles

Project-based ทุก concept จบเป็นไฟล์รันได้ · **Composable** ของ Day 1+2 ประกอบเป็น Day 3 · **Show & Tell daily** ปิดทุกวันด้วยโชว์คนละ 1 ชิ้น · **Dev-first** map TD เข้า mental model เดิม

วิธีอ่าน · แต่ละ session เรียงแบบนี้

🎯 goal · ⌚ run-of-show (ขึ้นก่อน — เห็นโครงเวลาทั้ง session เลย) · 1. ความสำคัญ · 2. ประเด็นที่ต้องพูด · 3. เนื้อหา + key scripts · 4. Output · ✂ เตรียมก่อน · 📄 handout · กล่องไฮไลต์ = หัวข้อเจาะลึก (มี (gap) กำกับให้)

Pedagogy dev vs designer: `teaching-approach.md` · ภาพรวมเวลา: `workshop-schedule.html` · workbook: `handout.html`

⚠ ไฟล์นี้ = **canonical source** ของเนื้อ session/scripts/run-of-show. **Derived views** ที่ต้องแก้ตามเมื่อเปลี่ยน session (ชื่อ/เวลา/script): `handout.html` + `workshop-schedule.html` + `mock-project-spec.md`

📌 Client Context — ดีดีเอ็กซ์พี (DDEXP) · มิ.ย. 2026

วัน/เวลา: 2 · 3 · 5 มิถุนายน 2026 · 10:30–18:30 (พักเที่ยง 12:30–13:30 · เบรก 15:30–15:45) · on-site · รันเป็นภาษาไทย

ผู้เรียน: 8 (dev-heavy: 4 Fullstack · 2 Frontend · 2 Designer) + 2 PM สังเกตการณ์ — frame TD สำหรับ dev ที่มาจาก code (ลิก/เร็วได้ ข้าม "node คืออะไร")

จุดแข็งทีม → ดึงมาใช้: เก่ง web/frontend/LINE → เน้นหนัก **S2.4 (TD as web server / phone-as-controller)** · มี service designer → ดึงเต็มที่ **S3.3 (installation/experience)** · ถ้าทีมเคยแตะ AI/ML → **S2.1 MediaPipe** เข้าง่าย

ธุรกิจ: engagement tech for events — โยงทุก session ว่า "build installation ลึกขึ้นเองได้" คือ value กับงานเขา

TouchDesigner คืออะไร

พื้นฐานที่ต้องเข้าใจก่อนลงมือ + ตารางเทียบกับเครื่องมือที่คุ้นเคย

TD คือ **real-time node-based dataflow engine** — วาง **graph** ของการแปลงข้อมูล แล้ว engine รันให้ทุกเฟรม เฉพาะจุดที่จำเป็น (ไม่ต้องเขียน render loop เอง). การทำงานคล้าย spreadsheet / React render tree / RxJS

TD ใช้ภาษา / เทคโนโลยีอะไร

engine + operator ทุกตัวเขียนด้วย **C++** รันบน **GPU** — งานหนัก (ภาพ, signal, 3D, render) ทำในขั้นนี้. ภาษาที่ใช้ร่วม (scripting) คือ **Python** ที่ฝังมาในตัว (embedded) — ไว้ทำ logic, control, glue. **ทำงานส่วนใหญ่ได้โดยไม่ต้องเขียน Python** แค่ต่อ node + กรอกค่า; งานจริงจังค่อยใช้ Python ช่วย. นอกจากนี้เขียน **GLSL** (shader บน GPU) ได้ และต่อ **custom operator** ด้วย **C++** ได้

Python อยู่ตรงไหน	ทำอะไร	เทียบ dev
Parameter expression	Python บรรทัดเดียวในช่อง param เช่น <code>op('audio') ['level']*200</code>	inline binding <code>{count*2}</code>
DAT (Text/Table)	ที่อยู่ของ script หลายบรรทัด / โมดูล	ไฟล์ <code>.js</code> ของคุณ
Callback (CHOP/DAT Execute)	รัน Python เมื่อเกิด event (ค่าเปลี่ยน, ปุ่มกด)	<code>onChange</code> / event handler
Script CHOP/TOP/SOP	operator ที่ output มาจาก Python/NumPy ของคุณ	custom function คำนวณ
Extension	Python class ผูกกับ COMP — state + method	component controller / class

rule of thumb: node สำหรับ dataflow/signal · Python สำหรับ logic, control flow, event, state, orchestration

หลักที่ dev ต้อง internalize

- **Cook = pull-based / lazy.** node จะ cook เมื่อมี 2 อย่าง: (1) **มีคนปลายทางขอ data** (viewer เปิด / output node / export) และ (2) **มีเหตุให้ cook** (input/param เปลี่ยน หรือเป็น time-dependent เช่น noise/วิดีโอ/เสียง). → ถึงที่ไม่มีใครใช้ = ฟรี. (เหมือน spreadsheet คำนวณเฉพาะ cell ที่กระทบ / React reconcile เฉพาะที่เปลี่ยน)
- **Operator = typed data families:** TOP=ภาพ/GPU · CHOP=สัญญาณ/ตัวเลข stream · SOP=3D geo · DAT=text/table/script · COMP=container/module. ต่อภายใน family เดียวกัน ข้าม family ต้องมี operator แปลง
- **Reactive ไม่ใช่ imperative** — ไม่สั่ง "ทำ 1 แล้ว 2" แต่ **ประกาศความสัมพันธ์** ("รัศมีวงกลม = ระดับ bass") แล้ว engine รักษาให้จริงตลอด = data binding

เทียบ p5.js — draw loop

p5 คุณ **เขียน loop เอง**: `function draw(){ r=bass*200; ellipse(x,y,r,r) }` รันทุกเฟรม. TD **ไม่มี draw() ที่คุณเขียน** — cook cycle คือ loop เอง. คุณวาง graph: Audio In → Analyze(bass) → Math(*200)

→ (bind) Circle → Render . p5 รันทุกบรรทัดทุกเฟรมไม่ว่าจะเปลี่ยนไหม; TD cook เฉพาะ node ที่ "มีคนขอ + มีเหตุ" = ชี้เร็วกว่าโดย default

dev concept → TD (ตารางแปลเร็ว)

ปกติเขียน...	ใน TD...
<code>draw()</code> / render loop	cook cycle ของ engine (ไม่ต้องเขียนเอง)
<code>setInterval</code> / timer	Timer CHOP , Beat CHOP , หรือ <code>me.time.frame % N</code>
<code>if/else</code>	Switch (เลือก input) / Logic CHOP / Python expression — ไม่มี "if node"
<code>for</code> รวน data	SOP/CHOP array ops, Replicator COMP , หรือ Python
event handler (<code>onClick</code>)	Execute DAT callback (CHOP/DAT/Panel)
ตัวแปรเก็บ state	output ของ operator (ค่าปัจจุบัน) · Storage (<code>store/fetch</code>) · extension attr
data binding <code>value={x}</code>	parameter reference expression หรือ export
import module · React component+props	.tox COMP / <code>mod()</code> · COMP + custom parameters
typed array / canvas+shader / mesh / JSON	CHOP / TOP / SOP / DAT
global <code>Date.now()</code>	<code>absTime.seconds</code> / <code>me.time</code> (time เป็น global)

mindset ที่ต้องพลิก + dev gotcha

- ไม่มี `main()` / entry point — graph + cook loop คือโปรแกรม · อยากให้อัปเดตทุกเฟรม = ทำ node ให้ time-dependent (ไม่เขียน loop)
- ไม่มี if/else node — branch คือ **เลือก data ที่ไหน** (Switch) ไม่ใช่กระโดดคำสั่ง
- state อยู่ใน operator/Storage **ไม่ใช่ตัวแปร Python module-level** (อันนั้น reset ได้ ไม่เข้ากับ dataflow)
- "ทุก node รันทุกเฟรม" = **ฉุด** (pull-based) · "เปิด viewer = ฟรี" = **ฉุด** (สร้าง cook request — ปิด viewer ที่ไม่ใช่ช่วย FPS)
- งานภาพ/signal อย่างถึงมาวนใน Python (ช้า) เก็บบน node (GPU) · ทำ .exe ไม่ได้ — ship ไฟล์ .toe

ที่มา verified: Derivative docs (First Things to Know · Cook · Operator) · Matthew Ragan · Interactive & Immersive HQ

DAY 1 — Foundation & First Interactive Sketch

🎯 ทุกคนมี interactive sketch ที่ใช้ได้จริงบนเครื่องตัวเอง — ปูพื้นแน่นแล้วได้ paint → audio → particle

1.0 Opening: Concept & What We'll Build 10:30–11:00 · 30m Concept 20 / Setup 10

🎯 เห็นภาพ 3 วัน + รู้ว่ากำลังจะสร้างอะไร + เครื่องพร้อมรัน

🕒 RUN OF SHOW

0–3	เปิด / welcome
3–18	concept + ตัวอย่างงาน + business framing + teaser mock
18–28	verify install ทดลองเครื่อง (designer/คนติดตั้งมือ)
28–30	segue

1. ความสำคัญ

ตั้งโจทย์ของ workshop ให้ชัด — สอน TD ผ่านการ build งานจริง (Draw → Scan → Touch) ไม่ใช่ทฤษฎีเพียงอย่างเดียว แล้ว fundamental ทุกอย่าง serve ปลายทางนั้น. dev ต้องเห็นปลายทางก่อนถึง engage

2. ประเด็นที่ต้องพูด

- TD = node-based visual programming สำหรับ real-time media (ไม่ใช่ video editor / game engine แต่ยืดแนวคิดทั้งคู่)
- ใช้จริงที่ไหน — installation, stage visual, projection mapping, interactive exhibit (โพรเจกต์ 3–4 ชั้น)
- เชื่อมกับธุรกิจของผู้เรียน: "งานที่ทีมทำอยู่ + TD = build installation/interactive ลึกขึ้นเองได้"
- DDEXP: engagement tech for events
- ปลายทาง 3 วัน = mock "Draw → Scan → Touch" · "วันนี้ปูพื้น+sketch, พรุ่งนี้ต่อ input, วันสุดท้าย build+ตั้งโชว์"

3. เนื้อหา + KEY SCRIPTS

ไม่มี script — orientation + verify install. เช็ค: TD เปิดได้ · license non-commercial · webcam/audio เข้า · เปิด mock teaser 30 วิ แล้วปิด

4. OUTPUT

ทุกเครื่องเปิด TD ได้ + เห็น demo ปลายทาง + รู้ว่ากำลังจะไปไหน

✂ **เตรียม:** mock teaser .toe · slide ตัวอย่างงาน · checklist install

📄 **Handout:** ปก + "What we'll build" diagram + checklist

1.1 TouchDesigner Thinking: Operators & Logic 11:00–12:30 · 90m

🎯 อ่าน network เป็น · รู้ family ไหนทำอะไร · ต่อค่าข้าม operator ได้ (reference/export)

🕒 RUN OF SHOW

- 0–25 mini-lecture: families + dataflow + ref/export
- 25–60 live demo (สร้าง network ข้างๆ, อธิบาย ref vs export)
- 60–85 hands-on: LFO→export→Circle + expression อ่านค่า
- 85–90 ใครติดยกมือ + segue

1. ความสำคัญ

Session ที่ทุกอย่างหลังจากนี้ยืนอยู่บน. ไม่เข้าใจ dataflow + วิธีต่อค่า → interactive ไม่เกิดเลย. จุด map TD เข้า mental model เดิมของ dev (reactive pipeline + data binding)

2. ประเด็นที่ต้องพูด

- TD ไม่ใช่ timeline (อย่าง AE) — เป็น **dataflow** ที่ operator "cook" เมื่อ input เปลี่ยน = reactive
- 5 families = 5 ชนิดข้อมูล: **TOP** ภาพ(GPU) · **CHOP** สัญญาณ/ตัวเลข stream · **SOP** 3D · **DAT** text/table/Python · **COMP** container/UI
- เส้น = ส่ง data; **parameter** ก็ link ข้าม op ได้ โดยไม่ต้องลากเส้น → reference vs export
- segue: "เดี๋ยวเอา concept นี้ไปทำ paint ที่ขยับตามเมาส์จริง"

References / Exports — ต่อค่าข้าม operator (gap · สอนก่อนเพื่อน)

2 วิธีต่อค่าจาก operator หนึ่งไปคุม parameter ของอีกตัว โดยไม่ลากเส้น. **ข้อต่างที่ต้องจำ: Reference ใส่สูตรได้ · Export ใส่สูตรไม่ได้ (ส่งค่าดิบล้วน)**

	Reference (Expression)	Export
ทำยังไง	พิมพ์ Python expression ลงใน parameter	ลาก CHOP channel มาวางทับ parameter
ใส่สูตร / คณิต / logic	ได้ — คุณ บวก รวมหลายแหล่ง ใส่เงื่อนไขได้	ไม่ได้ — ค่าดิบตรงๆ อย่างเดียว
รับค่าจาก	อะไรก็ได้ (CHOP, param, DAT, เวลา absTime)	CHOP channel เท่านั้น

ใช้อันไหนเมื่อไหร่: ต้องแปลงค่า / คุณสเกล / รวมหลาย input / ใส่เงื่อนไข → **Reference** · แด่เอา channel ไปคุม param ตรงๆ ไม่ต้องแต่ง → **Export** (ลากเร็วดี) · ทำ UI controller / งานทีม → **Bind** (two-way ค้างทั้งสองทาง)

เทียบ dev: Reference = computed getter `get radius(){ return audio.low*2 }` (ใส่สูตรได้) · Export = ต่อ stream เข้า prop ตรงๆ `audio.low$.subscribe(v=>circle.radius=v)` (ค่าดิบ)

parameter มี 4 โหมด ดูจากสีช่อง: Constant=เทา · Expression(ref)=ฟ้า · Export=เขียว · Bind=ม่วง

Logic / conditionals — ทำไมหา if/else ไม่เจอ (gap · dev ตามแน)

Paradigm: ภาษาทั่วไป if/else = คมว่า *โค้ดบรรทัดไหนรัน*. TD เป็น dataflow — ทุก node cook ตลอด ไม่มี "ข้ามบรรทัด" → ไม่ branch การรัน แต่ **เลือก/กรองว่า data** เส้นไหนไหลผ่าน

คิดแบบโค้ด	ใน TD ทำด้วย
switch / if...elif เลือก 1 จากหลายทาง	Switch CHOP/TOP/DAT (index เลือก input) — นี่คือ if/else เวอร์ชัน TD
cond ? a : b	Switch หรือ expression a if cond else b ในช่อง parameter
x > 0.5 → true/false	Logic CHOP (threshold→0/1) · Math/Limit — TD ไม่มี comparison node ตรงๆ
a && b · !a	Logic CHOP (And/Or/Not/Xor)
if event: doX()	Trigger CHOP → CHOP Execute DAT (รัน Python)
logic ซับซ้อน / state / loop	เขียน Python — Script CHOP, CHOP/DAT Execute · if/else จริงอยู่ตรงนี้

ต้องเขียน Python ไหม? ไม่เสมอ — เลือกเส้นทาง/threshold/boolean ง่ายๆ ใช้ node พอ (Switch/Logic) · เขียน Python เมื่อเงื่อนไขซับซ้อนหลายชั้น มี state หรือต้อง loop

ได้ใช้จริง Day 2 (S2.1): "มืออยู่ในเฟรมไหม?" → confidence จาก MediaPipe → Logic CHOP (>0.5→1/0) → Switch สลับ scene idle/active = "if hand present" แบบ TD

3. เนื้อหา + KEY SCRIPTS

Demo: Circle TOP → Blur → Null , ขยับ param ดู cook ไหลขวา

```
# Reference / Expression - ใส่สูตรได้
op('lfo1')['chan1']      # ดึงค่า CHOP channel
op('base1').par.Speed * 2 # เอา custom param มาคูณ
absTime.seconds          # เวลา (วินาที)
```

Export: ลาก channel จาก LFO CHOP → drop ทับ parameter radius (ช่องเปลี่ยนเป็นสีเขียว = ถูก export คมอยู่)

shortcut: Tab สร้าง op · u ขึ้น · i เข้า COMP · h home · middle-click = info. **DRY:** วาง Null เป็น anchor ก่อน reference (แก้ network ไม่พัง = abstraction layer)

4. OUTPUT

network ที่ LFO ขยับ param ผ่าน export + อีกตัวอ่านผ่าน expression · ผู้เรียนอ่าน network ออก บอกทิศ data ได้ · เชฟ .toe ตัวแรก

✂ **เตรียม:** demo .toe เปล่า · slide "5 families" · cheat sheet shortcut

📄 **Handout:** Operators & parameter modes + ตาราง ref vs export + ช่องจัด

1.2 Tutorial 1: Visual Interactive (Mouse → Feedback) 13:30–14:45 · 75m

🎨 paint app ที่เมาส์วาดแล้วเส้นค้าง — เข้าใจ feedback loop + ใช้ ref/export จริง

🕒 RUN OF SHOW

- 0–10 concept feedback
- 10–35 live demo (สร้าง paint ที่ละ node)
- 35–70 hands-on (ทำ + ปรับ brush/decay; designer จูนสี)
- 70–75 quick share

1. ความสำคัญ

Sketch แรกที่ทำเอง — ใช้ ref/export จาก 1.1 ทันทึ. **Feedback loop** จะกลับมาใช้ตอน particle trail + drawing accumulation (Day 3 รูปวาดค้างบนจอ = กลไกเดียวกัน). เริ่มด้วยเมาส์ (ยังไม่แตะกลอง) เพราะ input ตัวเดียว ตัวแปรน้อย ดีกับง่าย — โฟกัส TD logic ก่อน

2. ประเด็นที่ต้องพูด

- **Feedback loop คืออะไร:** เอา output ของ "เฟรมก่อน" ป้อนกลับมาผสมกับเฟรมนี้ → ภาพมี "ความจำ" สะสมขึ้นเรื่อยๆ. เทียบ dev: คล้าย state ที่ accumulate ทุก tick แต่ state เป็น "ภาพทั้งภาพ" (texture) ไม่ใช่ตัวเลขเดียว
- **ทำไมต้องมี Composite/Over :** ต้องเอา brush อันใหม่ "วางทับ" ภาพเก่าที่ค้างอยู่ — Over = alpha composite (ใหม่บนเก่า). ถ้าไม่ composite จะเห็นแค่ brush ตำแหน่งล่าสุด ไม่ค้าง
- **decay / fade:** ก่อนป้อนกลับ คุณภาพด้วยค่า < 1 (เช่น 0.95) ทุกเฟรม → เส้นเก่าจางหายช้าๆ. ไม่ลด = ค้างถาวรจืดเต็ม · ลดมาก = หางสั้น · ลดน้อย = หางยาว
- เทคนิคนี้เรียก **render-to-texture** — ภาพถูกวาดวนกลับเข้าตัวเอง ทำได้ด้วย node แค่ 2–3 ตัว
- segue: "ตอนนี้ input เป็นเมาส์ — เปลี่ยนเป็นเสียง (1.3) หรือมือ (Day 2) ได้ ด้วยโครงเดิม"

💡 เล่าให้เห็นภาพ (ไว้พูด / ทำ SLIDE)

นึกถึงกระดานที่ไม่เคยลบ — ทุกเฟรมเราวาดจุดใหม่ลงไป แล้วทั้งกระดานจางลงนิดนึง. จุดที่เพ่งวาดสว่างสุด รอยเก่าค่อยๆ จางเป็นหาง. ภาพปัจจุบัน = "ภาพเมื่อครั้งที่จางลง + ของใหม่ที่เพ่งวาด"

จุดที่ต้องเน้น: เส้นที่วนกลับไม่ทำให้ค้าง เพราะ Feedback TOP หยิบ "ภาพเฟรมก่อน" มา ไม่ใช่ภาพที่กำลังวาดอยู่ — เลย์ไมวนตาย. คนมักกังวลตรงนี้ พูดให้ชัดแล้วสบายใจ

slide ที่ควรมี: diagram วง feedback 1 รูป · before/after เปิด-ปิด decay · ตาราง decay value → ความยาวหาง

3. เนื้อหา — DEMO ทีละขั้น (พูดไปทำไป)

Chain: Mouse In CHOP → Circle TOP (brush) → Composite(over) ← Feedback TOP → Level($\times 0.95$) → Out

1. Mouse In CHOP → ได้ tx/ty ตำแหน่งเมาส์ (normalized) — "input ตัวแรกของเรา"
2. Circle TOP เป็น brush → ref ตำแหน่งจากเมาส์ (ของ 1.1)
3. Composite TOP (operation=Over) → ผสม brush กับภาพ feedback
4. Feedback TOP → ชี source ไปที่ output ของ Composite (จุดที่ "วน")
5. Level TOP (Opacity ~ 0.95) คั่นก่อนป้อนกลับ = decay

```
op('mousein1')['tx'] # brush Translate X
op('mousein1')['ty'] # brush Translate Y
```

ปุ่มเล่น: decay 0.90–0.99 (หางสั้น→ยาว) · brush size/สี/glow = designer จูน · เสริม: Slope CHOP ให้ brush โตตามความเร็วเมาส์

- "feedback วงกลับ = loop 'ไม่จบไหม?' → 'ไม่ Feedback TOP อ่าน "เฟรมก่อน" (1-frame delay) ไม่ใช่เฟรมปัจจุบัน
- "ภาพค้างไปเรื่อยๆ เมมเต็มไหม?" → ไม่ เป็น texture ขนาดคงที่ (res เดียว) เขียนทับทุกเฟรม
- "reset ยังไง?" → Feedback TOP มีปุ่ม reset (pulse) ล้าง buffer — Day 3 เอาไปทำปุ่ม reset งาน
- "ต่างกับ Cache/Trails TOP ไหม?" → มีหลายวิธี; feedback ยืดหยุ่นสุด เพราะจะ composite อะไรก่อนป้อนกลับก็ได้

4. OUTPUT

paint sketch: ลากเมาส์ = เส้นเรืองแสงค่อยจาง · เข้าใจ feedback loop ที่จะ reuse (particle trail + รูปค้าง Day 3) · เชฟ .toe

✂ **เตรียม:** demo paint .toe (ทำเสร็จ 1 + แปลง 1 ไวส์ร่างสด) · slide "feedback = render-to-texture loop" + diagram วง

📄 **Handout:** node diagram paint + ตาราง decay value + ช่องจัด

1.3 Tutorial 2: Audio Interactive (Mic → Visual) 14:45–15:30 · 45m

Concept 8 / Demo 17 / Hands-on 15 / Share 5

🎧 เอาเสียงจริงขึ้นภาพ — CHOP เป็น input stream + audio analysis

🕒 RUN OF SHOW

- 0–8 concept level/spectrum
- 8–25 demo
- 25–40 hands-on (เสียง → ขั้ว paint/วงกลม)
- 40–45 share

1. ความสำคัญ

Input ตัวที่สอง (เสียง) — ตอกย้ำ "อะไรก็เป็น CHOP ได้ แล้ว export ไปขั้ว param". audio-reactive = hello-world ของ TD, เร็วและ rewarding

2. ประเด็นที่ต้องพูด

- เสียง → ตัวเลข: Audio Device In → level (ดังแค่ไหน) หรือ spectrum (ความถี่ไหนแรง)
- level = 1 ค่า · spectrum = หลายค่า (bass/mid/treble) · "pattern เดียวกับเมื่อก็ แต่เปลี่ยนเส้นทาง"

3. เนื้อหา + KEY SCRIPTS

Audio Device In → Analyze(RMS) → Lag → export · spectrum: Audio Spectrum CHOP · normalize ด้วย Math CHOP (range 0..1) ก่อนใช้

4. OUTPUT

ภาพเต้นตามเสียง (ตบมือ/พูด → visual ขยับ) · เห็น pattern input→analyze→export→visual ขั้ว · เชฟ .toe

✂ **เตรียม:** demo audio-reactive .toe · เช็ค mic

📄 **Handout:** chain audio + Analyze vs Spectrum

1.4 Tutorial 3: First Particle Sketch 15:45–17:45 · 120m Concept 20 / Demo 40 / Hands-on 50 / Share 10

🎯 particle ที่ react ได้ + เข้าใจ instancing และ 3D render pipeline

🕒 RUN OF SHOW

0–20	concept instancing + 3D pipeline
20–60	live demo (Grid→instance→render ทีละขั้น)
60–110	hands-on (instance + ชับเสียง; คนไฉลอง GLSL)
110–120	share

1. ความสำคัญ

ขั้นสูงสุดของ Day 1 และเป็น หัวใจของ **mock**. เทคนิคนี้ Day 3 จะ reuse (เปลี่ยนต้นทางจาก noise → รูปวาด). มีเวลา
มากที่สุด ลงให้แน่น

2. ประเด็นที่ต้องพูด

- particle ใน TD ไม่ใช่ของวิเศษ = "วาด mesh ขึ้นเดียวซ้ำหลายที" = instancing
- ของ 3D ต้องมี render pipeline · ต้นทางตำแหน่ง instance มาจาก CHOP/SOP/**TOP** — วันนี้ noise, วันสุดท้ายเป็น
พิกเซลรูปวาด (TOP)
- segue: "พรุ่งนี้เอามือมาดันฝูง particle นี้"

Instancing & 3D Render Pipeline (gap · หัวใจของ particle)

particle ใน TD ทำด้วย instancing. อยากรู้ได้ทรงกลม 50,000 ลูก ถ้าสร้างทีละลูก = เครื่องตาย. **Instancing = วาด mesh** ดันแบบขึ้นเดียว แล้วส่ง GPU บีบเข้าเป็นพื้น-หมื่นตัว ในการวาดครั้งเดียว (1 draw call) → 50,000 ลูก ดันทุนเกือบเท่าลูกเดียว

กลไก: mesh ดันแบบ 1 อัน (เช่น Sphere เล็ก) + "ตาราง" ของ instance ที่บอกค่าต่อตัว: ตำแหน่ง (tx ty tz), หมุน, ขนาด, สี. 1 sample ในแหล่งข้อมูล = 1 instance. เลือกว่าช่องไหนคุณอะไร ในหน้า **Instance page** ของ **Geometry COMP**

แหล่งข้อมูลของ instance มาได้ 3 ทาง: **CHOP** (channel = ค่าต่อ instance) · **SOP** (จุดของ geometry) · **TOP** (พิกเซลของภาพ) — ตัว **TOP** นี้แหละคือสะพานไป "รูป → particle" ของ Day 3

โยงเข้า particle ของเรา: วันนี้ตำแหน่งมาจาก noise/grid (พุ่งลอยๆ ชับเสียง) · Day 3 เปลี่ยน source เป็น **TOP** ของรูปที่ scan มา → แต่ละพิกเซลที่มีสี = ตำแหน่ง+สีของ 1 instance → รูปวาดประกอบขึ้นเป็นฝูง particle · แล้วมี (Day 2) ดันให้กระจาย

3D render pipeline ขั้นต่ำ: Geometry COMP + Camera COMP + Light COMP → Render TOP → ภาพออกจอ (ของ 3D ต้องผ่าน Render TOP ก่อนถึงกลายเป็นภาพ 2D เสมอ)

เทียบ dev: instancing = `InstancedMesh (Three.js) / gl.drawArraysInstanced` — คนเคย WebGL เก๊ททันที. ไม่มี instancing = `draw()` 50,000 ครั้ง (CPU คอขวด); มี = 1 draw call บน GPU

gotcha (ชาว TD เตือน): ① ต้อง toggle เปิด **Instancing** บนหน้า **Instance** ของ **Geo COMP** ก่อน ไม่จันไม่ขึ้น · ② อย่าเอา SOP จุดเยอะๆ เสียบ source ตรงๆ (CPU คำนวณทุกเฟรม) — แปลงเป็น CHOP/TOP ก่อน · ③ อย่าใช้ Copy SOP ทำซ้ำ (นั่นรันบน CPU) — ใช้ instancing เสมอ

Method fork: 3D instanced (ทำตามนี้) ต้องมี render pipeline · ถ้าเลือก 2D GPU particle จะไม่ต้องแตะ 3D — **Wave** เลือก method ตัดสินตรงนี้

3. เนื้อหา + KEY SCRIPTS

pipeline: Geometry COMP (Sphere เล็ก) + Camera + Light → Render TOP · เปิด **Instancing** บน Geo COMP ซึ่ source ไป CHOP

ตำแหน่ง: Grid SOP (50×50) → SOP to CHOP (P0 P1 P2 = tx ty tz) หรือ Noise CHOP

```
# ขยับ instance ตามเวลา+เสียง (CHOP Math/Expression)
me.inputVal + op('audio_level')['chan1'] * 0.5
```

per-instance scale/count ขยับด้วย audio (export จาก 1.3) · เกรีน GLSL สำหรับคนไว (optional)

4. OUTPUT

ฝูง particle (instanced sphere) ขยับ/เปลี่ยนขนาดตามเสียง · เข้าใจ instancing + render pipeline = พร้อม reuse Day 3 · เชฟ .toe

✂ **เตรียม:** demo particle .toe · slide "instancing = GPU loop" + เทียบ `InstancedMesh` · fallback file

📄 **Handout:** diagram render pipeline + instancing source

1.5 Show & Tell — Day 1 17:45–18:30 · 45m Demo 30 / Discussion 15

🎯 ทุกคนโชว์ 1 sketch + เห็นงานเพื่อน + Wave จับ level ทิมไว้ปรับ Day 2

🕒 RUN OF SHOW

0–3	setup
3–33	โชว์ทีละคน ~3 นาที
33–43	Wave สรุป pattern + preview Day 2
43–45	ปิดวัน

1. ความสำคัญ

หลัก Show & Tell daily — บังคับให้ทุกคนมี "ของเสร็จ" 1 ชิ้น. Wave อ่าน level จริงเพื่อปรับ pace Day 2 (ใคร peer helper, ใครต้องประกอบ)

2. ประเด็นที่ต้องพูด

- "โชว์คนละชิ้น ไม่ต้องเพอร์เฟกต์ — เล่าว่าทำอะไร ติดตรงไหน"
- ชี้ pattern ร่วม: ทุกชิ้น = input→process→output · preview Day 2

3 + 4. เนื้อหา / OUTPUT

presentation + feedback. Wave จดว่าใครแข็ง (peer helper Day 3) ใครต้องประกอบ · ทุกคนมี ≥1 sketch โชว์ได้

📄 **Handout:** ช่อง reflection "วันนี้ทำอะไรได้ / ติดอะไร / พรุ่งนี้อยากลองอะไร"

DAY 2 — Input Systems & Integration

🎯 ต่อ input จริง (มือ + กล้อง) เข้า TD · เก็บข้อมูลลงไฟล์ · เห็นว่า TD ต่อเข้าระบบอื่นได้

2.0 Recap & Goal 10:30–11:00 · 30m Recap 20 / Setup 10

🎯 เชื่อม Day 1 → Day 2 · ทุกคนพร้อมเครื่อง + เข้าใจว่าวันนี้สร้าง "ระบบ input"

🕒 RUN OF SHOW

0–3	เปิด
3–20	recap + ถาม-ตอบค้างคา
20–30	verify plugin/webcam + segue

1. ความสำคัญ

ดึง momentum + ย้ำ mental model ก่อนเพิ่มความซับซ้อน · dev ที่หลุดเมื่อวานได้ตามทัน

2. ประเด็นที่ต้องพูด

- recap: ทุก sketch = input(เมาส์/เสียง)→export→visual
- วันนี้เปลี่ยน input เป็นของจริง: มือ(MediaPipe) + กล้อง+กระดาษ(ArUco) + เก็บไฟล์ · ปิดด้วย "TD ต่อระบบอื่นยังไง"

3 + 4. เนื้อหา / OUTPUT

เปิดไฟล์เมื่อวาน · เช็ค plugin MediaPipe + webcam · ทุกเครื่องพร้อม + เห็น roadmap

✂ **เตรียม:** เช็ค mediapipe-touchdesigner plugin ลงครบทุกเครื่อง (สำคัญ — ทำก่อนเริ่ม)

2.1 Body & Hand Tracking (MediaPipe → CHOP) 11:00–12:30 · 90m

Concept 15 / Demo 30 / Hands-on 40 / Share 5

🎯 ตั้งตำแหน่งมือ/ตัวจากกล้องเป็น CHOP แล้วจับภาพ

🕒 RUN OF SHOW

- 0–15 concept MediaPipe + normalized coords
- 15–45 demo (วาง plugin → map นิ้ว → ชับ brush)
- 45–85 hands-on (มือจับ particle/paint)
- 85–90 share

1. ความสำคัญ

Input ตัวเอกของ "Touch" ใน mock. ใช้ **mediapipe-touchdesigner plugin** (Torin Blankensmith) รัน MediaPipe ใน TD ผ่าน embedded Chromium — ไม่ต้องเขียน Python. ออกมาเป็น CHOP = ต่อ pattern เดิมได้ทันที. ถ้าทีมเคยแตะ **AI/ML จะเข้าง่าย** 📌 **DDEXP: ใช้**

2. ประเด็นที่ต้องพูด

- MediaPipe = ML model ของ Google หา landmark มือ/ตัว/หน้า จากภาพกล้อง
- plugin → output เป็น CHOP channel (ปลายนิ้ว tx/ty/tz) = "input stream อีกตัว เหมือนเมาส์/เสียง"
- ค่าเป็น normalized 0..1 ต้อง map · ต้อง Lag/Filter ลด jitter
- 🔗 **ได้ใช้ logic จาก S1.1:** "มือโผล่ในเฟรมไหม?" → confidence channel → **Logic CHOP** (threshold→0/1) → **Switch** สลับ idle/active = "if hand present" แบบ TD (ไม่มี if node)

3. เนื้อหา + KEY SCRIPTS

```
op('handtrack')['index_tip:tx'] - 0.5 # map 0..1 → -0.5..0.5
0.5 - op('handtrack')['index_tip:ty'] # flip Y
# Lag/Filter CHOP ลด jitter · Slope CHOP = ความเร็วมือ (แรงผลักที่หลัง)
```

ชื่อ channel ขึ้นกับ version ของ plugin — เช็ค viewer

4. OUTPUT

มือขยับหน้ากล้อง → brush/particle ตาม · เชื่อม MediaPipe→CHOP→visual ได้ · เซฟ .toe

✂ **เตรียม:** plugin ลงครบ · demo .toe · lighting ห้องพอให้
กล้องเห็นมือ

📄 **Handout:** chain MediaPipe→CHOP + coordinate
mapping

2.2 Computer Vision (Webcam + ArUco → Homography) 13:30–14:45 · 75m

Concept 15 / Demo 30 / Hands-on 25 / Share 5

🔗 scan รูปจากกระดาษให้ตรง (ตัด perspective) ด้วย ArUco + OpenCV

🕒 RUN OF SHOW

- 0–15 concept homography + ArUco
- 15–45 demo (Script TOP + warp, อธิบายโค้ด)
- 45–70 hands-on (dev เล่นโค้ด · ถือกกระดาษ/จัดกล้อง)
- 70–75 share

1. ความสำคัญ

ชั้น **custom** สุด นอกตำรา TD = ส่วนเสียงสุดของ mock → allocate mentoring เยอะ. จุดที่ dev ได้เขียน Python จริงใน Script TOP — flutter สาย code

2. ประเด็นที่ต้องพูด

- ปัญหา: กล้องมองกระดาษเฉียง → ภาพบิด. ต้อง "ดึงให้ตรง" = **homography / perspective warp**
- ArUco = marker 4 อันที่มุมกระดาษ → หาพิกัด 4 มุม → คำนวณ transform → warp เป็นสี่เหลี่ยมตรง
- Script TOP = เขียน Python (OpenCV) ได้ — "นี่คือที่ที่พวกคุณเขียนโค้ดเต็มที่"

ArUco scan — ทำให้ติดจริง (อ่านก่อนสอน) (deep-dive · verified TD 2023.1+ / OpenCV 4.8)

✅ **ข่าวดี:** TD 2023.1+ มี **OpenCV 4.8 + contrib + Python 3.11** มาให้แล้ว — `import cv2` ได้เลย ไม่ต้องลงอะไร (เช็ค: `print(cv2.__version__)` → 4.8.0). ถ้าใครเผลอลอง `opencv-python` ทับ → `cv2.aruco` หายทันที

⚠️ **แก้ bug ที่เคยเขียนไว้:** `numpyArray(delImageData=True)` **ไม่มีจริง** — ใช้ `numpyArray()` เฉยๆ (signature จริง = `numpyArray(delayed=False, writable=False)`)

API ต้องเป็นของใหม่ (OpenCV ≥4.7): `getPredefinedDictionary` → `DetectorParameters()` → `ArucoDetector(dict,params)` → `detectMarkers`. โค้ดเก่าจากเน็ต (`Dictionary_get`, `DetectorParameters_create`) **พังบน 4.8** — error ที่จะเจอ: `module 'cv2.aruco' has no attribute 'Dictionary_get'`

setup กระดาษ: print marker 4 อัน ID 0/1/2/3 = บนซ้าย/บนขวา/ล่างขวา/ล่างซ้าย · ใช้ **DICT_4X4_50** (cell ใหญ่ detect ง่ายที่ระยะ webcam) · ต้องมี **quiet zone** (ขอบขาวรอบ marker) ไม่จัน detect ไม่ติด · กระดาษแบน แสงนุ่มไม่สะท้อน · ใช้ **center ของ marker** เป็นจุด (rotation-stable)

2 จุดพึงบอยใน Script TOP: ① TD ภาพ origin **ล่างซ้าย**, OpenCV **บนซ้าย** → ต้อง `np.flipud` ทั้งขาเข้า-ออก (ลิม = ภาพกลับหัว/มิเรอร์) · ② TD = float32 0..1 RGBA, OpenCV = uint8 0..255 → แปลง dtype + **อย่าเผลอ convert BGR (R-B จะสลับ)** — pipeline นี้ใช้ RGB ตลอด

3. เนื้อหา + KEY SCRIPTS (VERIFIED · TD 2023.1+ OPENCV 4.8)

Video Device In TOP → Script TOP (ArUco+warp) → out · ตั้ง Script TOP resolution = 1024×1024 · ไฟล์เดิม + ตัว generate marker อยู่ที่ aruco/ (เปิด/รันได้เลย)

```
import numpy as np, cv2

DICT = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_4X4_50)
_det = cv2.aruco.ArucoDetector(DICT, cv2.aruco.DetectorParameters()) # build ครั้งเดียว
W = H = 1024
DST = np.float32([[0,0],[W-1,0],[W-1,H-1],[0,H-1]]) # TL,TR,BR,BL
_last = np.full((H,W,3), 0.15, np.float32) # fallback: ภาพ scan ล่าสุด

def order_by_id(corners, ids): # คำนวณ [TL,TR,BR,BL] = ID 0,1,2,3 (ใช้ center)
    if ids is None: return None
    c = {int(i): mc.reshape(4,2).mean(0) for mc,i in zip(corners, ids.flatten())}
    if not all(k in c for k in (0,1,2,3)): return None
    return np.float32([c[0],c[1],c[2],c[3]])

def onCook(scriptOp):
    global _last
    src = scriptOp.inputs[0].numpyArray() # float32 RGBA 0..1, origin ล่างซ้าย
    img = np.flipud(src) # flip -> origin บนซ้าย (OpenCV)
    u8 = (np.clip(img[:,:,:3],0,1)*255).astype(np.uint8)
    gray = cv2.cvtColor(u8, cv2.COLOR_RGB2GRAY)
    corners, ids, _ = _det.detectMarkers(gray)
    pts = order_by_id(corners, ids)
    if pts is None: # marker ไม่ครบ 4 -> คำนวณภาพล่าสุด
        scriptOp.copyNumpyArray(_alpha(np.flipud(_last))); return
    M = cv2.getPerspectiveTransform(pts, DST)
    warp = cv2.warpPerspective(u8, M, (W,H)).astype(np.float32)/255
    _last = warp
    scriptOp.copyNumpyArray(_alpha(np.flipud(warp))) # flip กลับ + ใส่ alpha

def _alpha(rgb):
    a = np.ones((*rgb.shape[:2],1), np.float32)
    return np.ascontiguousarray(np.concatenate([rgb,a],2))
```

4. OUTPUT

กล้องเล็งกระดาษมี marker → ภาพถูกดึงให้ตรงใน TD · เลียงกระดาษแล้วภาพยังตรง = homography ทำงาน · เซฟ .toe

TEST ก่อน WORKSHOP + FALLBACK (สำคัญ)

15 นาที pre-flight: print marker ติดมุมกระดาษ → `print(cv2.__version__)` ใน Textport (อยากเห็น 4.8.0) → ต่อ Video Device In → Script TOP → ใส่ debug `print(sorted(int(i) for i in ids.flatten()))` อยากเห็น `[0,1,2,3]` นึงๆ → เอา debug ออก เห็นภาพ flatten = ผ่าน

Fallback 3 ชั้น (อยู่ในโค้ดแล้ว): ① marker <4 อัน → คำนวณภาพ scan ล่าสุด ไม่ตำวบ · ② detect ไม่ติดสัด → manual 4-corner pick (warp ไม่สนว่าจุดมาจากไหน แก่แค่ pts) · ③ จุกเงิน → เปิดรูป pre-warped จาก Movie File In เล่นบทเรียนต่อ · **แสงนุ่มสม่ำเสมอ = ตัวแปรสำคัญสุด** (พึ่งเพราะแสงมากกว่าโค้ด)

✂ **เตรียม:** print marker 0-3 (มี quiet zone) · demo .toe · ไฟล์ aruco/ · ไฟ LED แสงนุ่ม · รูป pre-warped สำรอง

📄 **Handout:** flow scan + โค้ด ArUco เดิม (copy ได้) + ตาราง gotcha (API เก่า/ใหม่, flip, dtype)

2.3 Data Capture (Save → File → Auto-load) 14:45-15:30 · 45m

Concept 10 / Demo 20 / Hands-on 10 / Share 5

🎯 save รูป scan ลงไฟล์ + TD โหลดไฟล์ใหม่กลับเองอัตโนมัติ

🕒 RUN OF SHOW

- 0-10 concept file pipeline
- 10-30 demo (save + Folder DAT + auto-load)
- 30-40 hands-on
- 40-45 share

1. ความสำคัญ

ปิด loop kiosk: วาด→scan→เก็บ→โหลดกลับ. file I/O + folder watch = pattern ที่ใช้กับงาน data-driven ได้ (dev relevant)

2. ประเด็นที่ต้องพูด

- save ภาพ: Movie File Out TOP (snapshot) หรือ `cv2.imwrite` · ตั้งชื่อด้วย timestamp
- folder watch Folder DAT + auto-load DAT Execute → "event-driven file pipeline เหมือน file watcher ที่พวกคุณเขียน"

3. เนื้อหา + KEY SCRIPTS

```
op('warped').save(f'scans/draw_{int(absTime.seconds)}.png')

# Folder DAT → DAT Execute (onTableChange)
def onTableChange(dat):
    latest = dat[dat.numRows-1, 'name'].val
    op('moviefilein1').par.file = f'scans/{latest}'
    return
```

4. OUTPUT

กดบันทึก → ไฟล์ PNG เกิด → TD แต่งโหลดภาพล่าสุดเอง · เชฟ .toe

✂ **เตรียม:** demo .toe save+watch · โฟลเดอร์ scans/

📄 **Handout:** chain save/watch + โค้ด (copy ได้)

2.4 Web → Interactive (TD as Server) 15:45-17:15 · 90m

 เอา web stack ที่ทีมถนัดมาใช้งาน interactive — TD เป็น server, มือถือคนดูคุม installation ผ่านเว็บ

🕒 RUN OF SHOW

- 0–20 concept: Web Render / Web Server / WebSocket / Web Client + pattern phone-controller
- 20–60 demo: Web Server DAT เสิร์ฟหน้าเว็บ → มือถือแตะ → ชับ particle (+ Web Render TOP เอาเว็บมาเป็น layer)
- 60–85 hands-on (มือถือตัวเองคุม visual ผ่าน WS; ต่อ backend/LINE ถ้าไฉ)
- 85–90 share

1. ความสำคัญ

Session ที่ทำให้ทีมเห็นว่า TD ต่อกับของที่ตัวเองถนัดได้. ถ้าทีมเก่ง web/frontend — แทนที่จะทิ้งแล้วเรียน TD ใหม่หมด, อันนี้ เอา superpower เดิมมาใช้ใน installation. pattern "มือถือคนดู → คุมงานผ่านเว็บ" = engagement tech ของจริง

📌 DDEXP: จุดแข็ง #1 = web/LINE — เน้นหนัก session นี้

2. ประเด็นที่ต้องพูด

- TD เป็น server ได้ — HTTP + WebSocket ในตัว (Web Server DAT)
- 4 ตัว: Web Render TOP (เว็บ→texture) · Web Server DAT (TD host หน้าเว็บ+WS) · WebSocket DAT (ต่อ backend) · Web Client DAT (เรียก REST/LINE API)
- เทียบ dev: Web Server DAT = Express + ws ผังใน TD · Web Render TOP = iframe/Electron ที่ output เป็น texture · WebSocket DAT = ws client · Web Client DAT = fetch()
- "OSC / MIDI / DMX / Serial ก็ pattern เดียวกัน (DAT/CHOP รับ-ส่ง) — ไม่ลงลึกวันนี้" (ลดจากเดิม)

★ Web ↔ TD — 4 ตัวหลัก + pattern เด็ด

Operator	ทำอะไร	เทียบ dev
Web Render TOP	รันเว็บ/Three.js/canvas สด → texture ใน TD	iframe / Electron window
Web Server DAT	TD เป็น HTTP+WebSocket server — เสิร์ฟหน้าเว็บ + รับ message	Express + ws
WebSocket DAT	TD เป็น client ต่อ WS server ภายนอก (backend, Mosquitto ws://9001)	new WebSocket()
Web Client DAT	ยิง HTTP GET/POST → REST / LINE API / CRM	fetch() / axios

Pattern เด็ด — phone-as-controller: มือถือเปิดหน้าที่ TD เสิร์ฟ → แตะ/ลาก → WebSocket → TD ชับ particle/scene → broadcast กลับทุกเครื่อง. คนดูหลายคนเล่นพร้อมกันผ่านมือถือ ไม่ต้องแจก controller = engagement tech for events

3. เนื้อหา + KEY SCRIPTS

```
# Web Server DAT callbacks – TD เป็น server
def onHTTPRequest(webServerDAT, request, response):
    response['statusCode'] = 200
    response['data'] = op('controlpage').text    # HTML ใน Text DAT
    return response

def onWebSocketReceiveText(webServerDAT, client, data):
    import json
    msg = json.loads(data)                      # จากมือถือ
    op('control').par.value0 = msg.get('push', 0) # ขั้ว parameter
    return
# broadcast กลับทุก client: webServerDAT.webSocketSendText(text)
```

```
# หน้าเว็บที่เสิร์ฟ (controlpage Text DAT)
<script>
const ws = new WebSocket('ws://' + location.host);
document.body.onclick = e => ws.send(JSON.stringify({push: 1.0}));
</script>
```

4. OUTPUT

มือถือเปิดเบราว์เซอร์ → ตะ → particle ใน TD ขยับ (phone คม installation จริง) · dev เห็นว่าเอา web stack ตัวเองมาต่อ TD ได้ · เซฟ .toe + control-page HTML

✂ **เตรียม:** demo .toe (Web Server DAT + particle) + control-page HTML สำเร็จ · เครื่องกับมือถืออยู่ WiFi เดียวกัน · เช็ก IP/firewall port

📄 **Handout:** 4 web operators + Web Server DAT callbacks + control-page HTML (copy ได้)

2.5 Show & Tell — Day 2 17:15–18:30 · 75m Demo 40 / Discussion 20 / Brief 15

🎯 โวระบบ input + capture + เตรียมหัวเข้า Day 3 (มอง mock เป็นขั้นเดียว)

🕒 RUN OF SHOW

- 0–3 setup
- 3–43 โหวทีละคน
- 43–58 discussion (ขึ้นส่วนต่อกันยังไง)
- 58–73 brief mock Day 3 + แบ่งทีม
- 73–75 ปิดวัน

1. ความสำคัญ

จุดบรรจบ — มีขั้นส่วน (มือ, scan, save, integrate) ครบ. ทำให้เห็นว่า "พรั่งนี้แค่เอามาต่อกัน" ไม่ใช่เริ่มใหม่ (composable)

2. ประเด็นที่ต้องพูด

- โขว์ระบบ input ของแต่ละคน · ชั่วทุกชั้น = บล็อกของ mock
- brief mock Day 3 ล่วงหน้า ให้ไปคิดข้ามคืน + แบ่งทีม

3 + 4. เนื้อหา / OUTPUT

demo + planning · ทุกคนมีระบบ input ≥ 1 ที่ทำงาน + เห็นภาพ Day 3 = ประกอบ ไม่ใช่เริ่มใหม่

 **Handout:** reflection + "ชิ้นส่วนของฉันที่จะเอาไปต่อ Day 3"

DAY 3 — Ship Your Mini Project

 ประกอบเป็น mock จริง → ทำ UI → คิดเรื่องการตั้งชื่อ → ติดตั้งจริง → นำเสนอ

3.0 Brief & Recap: the Mock 10:30–11:00 · 30m Brief 20 / Team setup 10

 ทุกทีมเข้าใจ spec mock + แบ่งงานในทีม

🕒 RUN OF SHOW

- | | |
|-------|----------------------------|
| 0–20 | brief spec + demo เป้าหมาย |
| 20–30 | แบ่งทีม/บทบาท + segue |

1. ความสำคัญ

ตั้งโจทย์วันสุดท้ายให้ชัด ป้องกันหลงทาง · ย้ำว่าใช้ของที่เข้ามาแล้ว 2 วัน

2. ประเด็นที่ต้องพูด

- spec: **Draw** → **Scan** → **Particle** → **Hand-interact** · ทุกทีมทำตัวเดียวกัน (baseline) · คนไวด่อ stretch (body pose)
- บทบาท: dev คม logic/integration · designer คมภาพ/template/นำเสนอ · timeline: build เข้า · UI+install ปลาย · present เย็น

3 + 4. เนื้อหา / OUTPUT

review ชิ้นส่วนที่จะต่อ: instancing(1.4) + scan(2.2) + save(2.3) + hand(2.1) · ทุกทีมรู้ว่าทำอะไร + ใครทำส่วนไหน

 **เตรียม:** spec mock 1 หน้า · diagram ชิ้นส่วน→mock

 **Handout:** spec + diagram pipeline + ช่องว่างแผนทีม

3.1 Build Time: Compose & Live Mentoring 11:00–12:30 · 90m Concept 15 / Build 70 / Share 5

🎯 ต่อขึ้นส่วนเป็น mock ที่ทำงาน + เรียน modularity (COMP/.tox)

🕒 RUN OF SHOW

- 0–15 concept compose + COMP/.tox
- 15–85 build (Wave + peer เดินช่วยทีละทิม — เน้น ArUco/instance ที่เสียง)
- 85–90 quick share สถานะ

1. ความสำคัญ

หัวใจ Day 3. drawing→particle ใช้ instancing จาก 1.4 (เปลี่ยน source เป็นรูปวาด). ทิม dev คิดเรื่อง architecture อยู่แล้ว — สอนห่อ COMP = กัน spaghetti + reuse ข้ามทิม

2. ประเด็นที่ต้องพูด

- ประกอบ: รูป scan(2.2/2.3) → source ของ instance (แทน noise ใน 1.4) → มือ(2.1) ดัน particle
- เปลี่ยนรูป→particle: sample พิกเซลที่มีสี → ตำแหน่ง+สีของแต่ละ instance · "อย่าสร้าง network ก้อนเดียวยักษ์ — ห่อเป็นโมดูล"

COMPs / .tox / Modularity — build like engineers (gap)

COMP (Component) = กล่องที่ห่อ network ทั้งก้อน. สร้างครั้งเดียว → ห่อ → เปิด custom parameter เป็น "หน้าบ้าน/API" → reuse ได้

TD	โลก code
COMP	class / module / component
Custom parameter	constructor args / props
.tox file	npm package / importable module
ลาก COMP หลายตัว	instantiate objects · DRY

ตัวอย่าง: particle system เป็น COMP ที่มี param color/count/speed → ทิม A และ B ใช้ COMP เดียวกัน ปรับ param ต่างกัน ไม่ต้อง copy-paste network

```
parent().par.Pushforce      # อ่าน custom param ของ COMP แม่ จากข้างใน
# ห่อ: เลือก network → RMB → Collapse Selected → COMP
# Custom Parameter (Component Editor) → ตั้ง param
# export: RMB COMP → Save Component .tox → ลากกลับเข้าโปรเจกต์ไหนก็ได้
```

3. เนื้อหา + KEY SCRIPTS

รูป→instance: Movie File In (รูป scan) → TOP to CHOP (sample พิกเซลเป็นตำแหน่ง/สี) → instance source ·
มือนัด: hand velocity (Slope จาก 2.1) → offset ตำแหน่ง instance

4. OUTPUT

mock: scan รูป → particle ตามรูป → ปิดมือ particle กระจาย · อย่างน้อย 1 ชั้นห่อเป็น .tox มี custom param · เชฟ .toe + .tox

✂ **เตรียม:** demo mock เต็ม · ชิ้นส่วน .tox สำรอง (scan.tox, particles.tox) · peer helper จาก Day 1

📄 **Handout:** diagram compose + วิธีห่อ COMP/.tox + custom param

3.2 TouchDesigner UI Essentials 13:30–14:30 · 60m Concept 12 / Demo 23 / Hands-on 20 / Share 5

🎯 control panel คมงานหน้างาน + presets — เนาะๆ พอใช้ได้ (dev มี UI framework เองอยู่แล้ว)

🕒 RUN OF SHOW

- 0–12 concept UI/custom param
- 12–35 demo (custom param + reset + slider)
- 35–55 hands-on (ทำหน้าคุม mock; designer จัด layout)
- 55–60 share

1. ความสำคัญ

งาน install ต้องมีหน้าคุม (เปิด/รีเซ็ต/ปรับค่า) โดยไม่แตะ network. สอนเบตามที่ research แนะนำ — dev เก่ง UI อยู่แล้ว

2. ประเด็นที่ต้องพูด

- คนคุมงานไม่ควรต้องงม network สด · TD UI = COMP (Container/Button/Slider) + อ่านค่าเข้า Panel CHOP
- custom parameter (จาก 3.1) = วิธีเร็วสุด · presets = Storage / parameter snapshot · "ไม่ต้องสว้ย แค่ใช้ได้ + reset ได้"

3. เนื้อหา + KEY SCRIPTS

```
# ปุ่ม reset (Button callback)
def onClick(panelValue):
    op('feedback1').par.resetpulse.pulse() # สร้าง feedback/particle
    return
```

panel: Container + Button/Slider → Panel CHOP อ่านค่า → export เข้า param · presets ผ่าน Storage

4. OUTPUT

control panel เล็ก: reset + 2–3 slider คมงาน · เชฟ .toe

✂ **เตรียม:** demo UI .toe · template container

📄 **Handout:** วิธีเปิด custom param + ปุ่ม reset code

3.3 Installation Thinking: Experience & Placemaking 14:30–15:30 · 60m

🎯 คิดงานเป็น "ประสบการณ์ในพื้นที่" ไม่ใช่แค่ของบนจอ — วางแผน install จริง

🕒 RUN OF SHOW

- 0–20 mini-lecture experience/placemaking + ตัวอย่างงาน install
- 20–45 discussion + ทีมวาดผัง install
- 45–60 แต่ละทีมเล่าผัง + segue install จริง

1. ความสำคัญ

ส่วน **value-add** ที่ **Wave (cultural futurist)** เติม — ไม่มีใน technical TD course. เปลี่ยน mindset จาก "ทำ effect" เป็น "ออกแบบประสบการณ์คน". ถ้ามี designer/UX ในทีม = ดึงจุดแข็งเขาเต็มที่

📌 DDEXP: designer 2 คน = service designer มีอาชีพ

2. ประเด็นที่ต้องพูด

- งาน interactive อยู่ใน "พื้นที่" ไม่ใช่แค่จอ — คนเดินเข้ายังงี้, รู้ได้ใจว่าเล่นได้, เล่นเสร็จแล้วงี้
- วงจร: **attract** → **invite** → **engage** → **release**
- placemaking — งานเปลี่ยนความหมายพื้นที่ · sightline, แสง, เสียง, การจราจรคน
- idle/attract state · ความทน (รันทั้งวัน, reset ง่าย, ไม่พังถ้าคนทำแปลกๆ) · "คิด UX ของพื้นที่จริงเหมือน UX ของ app"

3 + 4. เนื้อหา / OUTPUT

discussion + planning · โจทย์: ทีมวาดผังการตั้ง mock (กล่อง/จอ/projector/คนยืน/idle state) · ได้แผนตั้งงาน + mindset experience-first

✂ **เตรียม:** slide attract→release + ตัวอย่างงาน install · กระดาษ/ปากกาวาดผัง

📄 **Handout:** วงจร attract→release + checklist install + ช่องวาดผัง

3.4 Physical Prototyping & Live Install 15:45–17:30 · 105m

Build 30 / Install 45 / Optimize 20 / Buffer 10

🎯 ตั้ง mock เป็น installation จริง (cardboard + projector) + จูน performance ให้รันลื่น

🕒 RUN OF SHOW

- 0–30 ประกอบ station + ยิง projector
- 30–75 install + จูนหน้างาน (Wave/peer ช่วย)
- 75–95 optimization pass (หา/แก้จุดอืด + idle/reset)
- 95–105 buffer/แก้ปัญหา

1. ความสำคัญ

เอา digital ออกมาสู่ physical — cardboard scan station + ยิง projector จริง. งานรันสดหน้าคน FPS ห้ามตก

2. ประเด็นที่ต้องพูด

- ประกอบ cardboard scan station (ฐานกล่อง + กรอบกระดาษ) — craft + tech
- ยิง projector: resolution, aspect ratio, fullscreen (Window COMP) · reset/idle จากแผน 3.3

Optimization — กัน FPS ตกหน้างาน (gap)

งาน install รันสด → ห่วง frame rate. concept หลัก:

- Cook** — operator คำนวณใหม่เมื่อ input เปลี่ยน. อะไร cook ทุก frame โดยไม่จำเป็น = กิน FPS
- Resolution** — TOP ที่ 4K แพงกว่า 1080p 4 เท่า · ใช้ res ให้พอดี
- GPU vs CPU** — งานอยู่บน GPU (TOPs) เร็ว · บาง CHOP/SOP เป็น CPU คอขวดได้

เทียบ dev: profiling + เลี่ยงคำนวณซ้ำ (memoization) + เก็บ hot path บน GPU + คม frame budget (16.6ms ต่อ 60fps)

```
op('render1').cookTime          # cook time (ms)
# Perform Mode = F1 (fullscreen) · Performance Monitor = Alt+Y · Probe (palette)
```

จน: ลด res ที่ไม่จำเป็น · Null + selective cook · Cache/bypass สาขาไม่ใช่ · instance แทน copy (มีแล้ว) · idle state = timer ไม่มี input → กลับ attract loop

3. เนื้อหา + KEY SCRIPTS

fullscreen: Window COMP → ตั้ง projector → Perform Mode (F1)

4. OUTPUT

mock ตั้งเป็น installation จริง ยิง projector เล่นได้ · รันสั้น (รู้วิธีหา/แก้จุดอืด) · มี idle/reset

✳ **เตรียม:** cardboard + เครื่องมือ · projector (อันเล็ก office — verify) · webcam + extender · demo Perform/Probe

📄 **Handout:** checklist install + Window COMP/Perform + วิธีดู cook time + tips optimize

3.5 Final Presentation 17:30–18:30 · 60m Demo 35 / Retro 15 / How-to-continue 10

🎯 แต่ละทีมโชว์งานที่ตั้งจริง + retro + วิธีเรียนรู้ต่อเอง

🕒 RUN OF SHOW

- 0–5 setup
- 5–40 โชว์ทีละทีม ~8–10 นาที
- 40–55 retro รวม
- 55–60 how-to-continue + ปิด

1. ความสำคัญ

ปิด workshop ด้วยของจริงที่ "ship" แล้ว (เป้าหมายทั้งหมด) · ตอน Learning Outcome #5 — เรียนต่อเองได้

2. ประเด็นที่ต้องพูด

- แต่ละทีมโชว์ + เล่า: ทำอะไร, ติดอะไร, แก้อย่างไร, ต่อยอดกับงานตัวเองยังไง
- retro: 3 วันได้อะไร, อะไรยาก, อะไร surprising
- เรียนต่อ: docs (derivative.ca) · community (TD forum, Discord) · YouTube (elektronaut, Matthew Ragan) · AI-assisted (Claude/Gemini ช่วยเขียน Script TOP)
- "พวกคุณเริ่มโปรเจกต์เองได้แล้ว — นี่คือเป้าหมาย"

3 + 4. เนื้อหา / OUTPUT

presentation + reflection · ทุกทีมโชว์ installation ที่ทำงาน + รายการ resource · ถ่ายรูป/วิดีโอเก็บ portfolio

✂ **เตรียม:** projector/ลำดับโชว์ · slide resource · ถ่ายรูป/วิดีโอ

📄 **Handout:** resource เรียนต่อ + ช่อง retro + "โปรเจกต์ต่อไปของฉัน"

Appendix — Master Script Bank · Checklist · Shortcuts

รวม key scripts (illustrative — ปรับตาม version จริง)

```
# Reference/Expression (1.1)
op('lfo1')['chan1'] · op('base1').par.Speed*2 · absTime.seconds · parent().par.Pushforce
# MediaPipe (2.1)
op('handtrack')['index_tip:tx'] - 0.5
# Save + folder watch (2.3)
op('warped').save(f'scans/draw_{int(absTime.seconds)}.png')
# Reset (3.2)
op('feedback1').par.resetpulse.pulse()
# Performance (3.4)
op('render1').cookTime · Perform=F1 · Perf Monitor=Alt+Y · Probe(palette)
```

Pre-workshop install	Shortcut
TouchDesigner (non-commercial)	Tab สร้าง op
webcam + mic	u ขึ้น · i เข้า COMP · h home
mediapipe-touchdesigner plugin	middle-click = info
opencv-contrib-python (ArUco)	F1 Perform · Alt+Y Perf Monitor
(optional) paho-mqtt	scroll=zoom · middle-drag=pan